

# Introduction to Python

SyoSil, 2026

# Contents

- Background on Python
- Python basics
- Advanced Python concepts
- Object-oriented programming in Python
- Modular Programming in Python
- Working with Python environments

# Python Background

# Python Background

- Easy to read and write. Almost no boilerplate code required to get off the ground
- Python is an interpreted language, not a compiled language
- C and Java are examples of compiled languages.
  - To run, you must first compile (e.g. with gcc) and then execute
- Interpreted languages are run by an interpreter, a program that converts the code into instructions on-the-fly
  - Faster startup than compilation, but slower overall execution

# Python Basics

# Hello, Python

- No boilerplate code is required to execute Python code
- Save in `hello.py`
- Execute with `python`:

```
# Hello, world  
# Comments are written with #  
print("Hello, Python!")
```

```
""" Multi-line comments  
Are written with  
triple-quotes """
```

```
$ python hello.py  
  
Hello, Python!
```

# Python Basics / Types

- Python is a dynamically typed language
- Variables do not have explicit types, and the type of a variable can change during execution
- Python uses indentation to delimit blocks. Statements are auto-detected
  - No curly braces or semicolons required

```
a = "Hello, world" # String inferred
b = 1234           # int inferred
c = True           # Bool inferred
if a != b:         # Legal comparison
    a = b          # a is now an int
    c = 4.321      # c is now a float
```

# Python Basics / Types

- Variables are dynamically typed, but Python still uses datatypes internally
- `int`: Integers of **arbitrary width**
  - Not just 32-bit or 64-bit
- `float`: 64-bit floating-point numbers
  - Called `double` in C or Java
- `str`: Datatype for strings and characters
  - Python doesn't distinguish between `'` and `"`

```
myint = 10 << 100 #103 bits
myfloat = 84.42
str1 = "Hello"
str2 = 'World'
str3 = '!'
```

```
print(type(myint), type(myfloat),
      type(str1), type(str2), type(str3))
```

```
$ python types.py

<class 'int'> <class 'float'>
<class 'str'> <class 'str'>
<class 'str'>
```

# Python Basics / Arithmetic Operators

- `+`, `-`, `*` : Add / sub / mul. If used with int and float in the same expression, result is a float
- No increment/decrement operators `++`/`--`: Use `+=1` and `-=1`
- Division with `/`: Division with floating-point result, even if both operands are integers
- Division with `//` : Truncating division (round down), even if both operands are floating point
  - If either operand is floating point, the result is a floating-point, but the result is still truncated
- Power operator `**`: Raises one integer to the power of another integer
  - For floating point, use `pow(base, exp)`

```
a = 10
b = 3
c = 8
pi = 3.14

addsub = a + b - c
mul = a * c
mulf = a * pi #31.4, auto-cast to float

divFloat = a / b #3.333...
divInt = a // b #3

powerInt = 2**8 #256
powerFloat = pow(2.5, 3.14)
```

# Python Basics / Logic and Bitwise Operators

- Boolean constants in Python are True, False
- AND, OR, NOT operators have text representation and, or, not
- XOR operator is ^ (hat)
- Bitwise operators are &, | and ^
- Binary and hex values can be created by prepending 0b and 0x

```
t = True
f = False

x = t and f
y = (pi > 5) or (42 < 100) and (not False)

c1 = 0b10110101
c2 = 0xa2
r = (c1 & c2) | (12) ^ 0b1010100
```

# Python Basics / Printing

- The print function is used for all printing purposes
- Python f-strings allow for value embedding and changing the format
  - x: Printed as normal
  - y: Printed as 3-digit hex value with leading zeros
  - z: Printed using scientific notation and 2 digits after decimal point

```
print("Hello, World")  
  
x = 10  
y = 3  
z = x / y  
  
print(f"x={x}, y={y:03x}, z={z:.2e}")
```

```
$ python print.py  
  
Hello, World  
x=170, y=00d, z=1.31e+01
```

# Python basics / Lists

- Python equivalent to arrays, can contain any type
- `.append(item)` appends `item` to the list
- Lists can be sliced as `mylist[i:j]`
  - returns items `i` to `j`
  - including index `i`
  - excluding index `j`
- Negative index counts from the end of the list, e.g. `mylist[-1]` returns last item in list
- `len(mylist)` returns length of list

```
mylist = [1,2,3,4]
print(mylist)
mylist.append(5)
print(mylist[2:4])
print(mylist[-4:-2])
print(len(mylist))
```

```
$ python lists.py
```

```
[1, 2, 3, 4]
[3, 4]
[2, 3]
5
```

# Python Basics / Iteration

- For-statements differ from C/Java
- Not only iteration over arithmetic sequences, but over any sequence
- Range type allows for easy iteration over integer sequence
- Extra parameters allow you to change starting point and index step
  - `range(start, stop, step)`
- Also supports backwards iteration if `start > stop` and `step` is negative

```
for item in mylist
    print(i, end=" ")

for i in range(5):
    print(i, end=" ")

for j in range(1, 10, 3):
    print(j, end=" ")
```

```
$ python iterate1.py
```

```
1 2 3 4
0 1 2 3 4
1 4 7
```

# Python Basics / Iteration

- While loops work as you expect
- Python does not support do-while loops

```
i = 10
while i > 6:
    print(i)
    i -= 1
```

```
$ python iterate2.py
```

```
10
9
8
7
```

# Python Basics / Iteration

- Continue statement finishes current loop iteration, moves straight to next loop iteration
- Break statements break out of loop

```
for i in range(10):  
    if i % 3 == 0:  
        continue  
    print(i, end=" ")  
    if (i >= 8):  
        break
```

```
$ python iterate3.py
```

```
1 2 4 5 7 8
```

# Python Basics / Flow Control

- If/elif/else statements don't require parentheses around arguments
  - Note special keyword for elif-clause
  - Parentheses are not required around the statement being checked
- Switch-statements are called match-statements in Python
  - Multiple clauses are separated by |
  - For long expressions, use indented block
  - Default case is expressed by \_
  - If no default case, simply falls through

```
if x > 4: #No brackets around clause
    print("A")
elif (b < 10): #With brackets
    print("B")
else:
    print("C")

match fruit:
    case "orange": print("Juice")
    case "banana":
        print("Delicious!")
        print("Yellow!")
    case "apple" | "pear": print("Same")
    case _: print("Any other fruit")
```

# Python Basics / Functions

- Functions are declared with def keyword
- Return value is specified with 'return' keyword. If not present, None-value is returned
  - Special Python value representing the absence of a value
- Functions can return more than one value
  - Returns an N-tuple.

```
def myfun(a,b,c):  
    return a + b * c  
  
def foobar(a, b, c):  
    return (a + b, a + c)  
  
print(myfun(1,2,4))  
(X, Y) = foobar(1,2,3)  
print(X, Y)
```

```
$ python fun.py
```

```
9  
3 4
```

# Python Basics / Dictionaries

- Dictionaries allow you to create key:value associations
  - Known as a Map or Associative Array in many other languages
- Each key can have an associated value
  - If key does not exist when inserting, create key
  - If key does exist when inserting, overwrite old value

```
dict = {}  
dict["apple"] = 3  
dict["wood"] = -2  
dict["water"] = 5  
print(dict)  
  
dict["wood"] = 10 #Overwrite old value  
print(dict)
```

```
$ python dicts.py
```

```
{'apple': 3, 'water': 5, 'wood': -2}  
{'apple': 3, 'water': 5, 'wood': 10}
```

# Python Basics / Dictionaries

- Check if a key exists in a dictionary with the 'in' keyword
- Function 'get' allows you to specify a default value to return if no value is mapped

```
# Same items as before
if "stone" in dict:
    print(dict["stone"])
else:
    print("Key 'stone' not found")

# Equivalent statement
print(dict.get("Stone", "Key 'stone' not found"))
```

```
$ python dicts2.py
```

```
Key 'stone' was not found
Key 'stone' was not found
```

# Python Basics / Dictionaries

- Dictionaries can be iterated over, just like lists
- Must specify whether you want the items, just the keys or just the values

```
# same dict as previous slides
for (key, val) in dict.items():
    print(f"I have {val} {key}")

for key in dict:
    # Do something

for val in dict.values():
    # Do something
```

```
$ python dicts.py
```

```
I have 3 apple
I have 10 wood
I have 5 water
```

# Object-oriented programming in Python

# Object-oriented programming in Python

- Classes and OOP in Python
- Simple class definitions
- Inheritance
- Instance vs. static methods/variables

# OOP in Python / Introduction

- Python supports object-oriented programming
- Classes are defined with the `class` keyword
- The constructor is named `__init__(self,...)`
- **Important:** All instance methods must have `self` as first parameter
  - Corresponds to 'this' in languages like Java
  - `self` is passed implicitly; you don't pass it (notice that `call_name` has no arguments given)

```
class Dog():  
    def __init__(self, name):  
        self.name = name  
  
    def call_name(self):  
        print(f"Come here, {self.name}!")  
  
fido = Dog("Fido")  
fido.call_name()
```

```
$ python oop.py
```

```
Come here, Fido!
```

# OOP in Python / Introduction

- All variables and functions are (technically) public!
- Internal “private” variables and methods are prefixed with `_`
  - Does not hide or make the method/variable private, but by convention indicates that it should not be called from outside that class

```
class Cat():
    def __init__(self, name, age):
        self._name = name # "Private"
        self.age = age # "Public"

    def _gen_info(self):
        return f"{self._name} is {self.age} years old"

    def info(self):
        return self._gen_info()

garfield = Cat("Garfield", 8)
garfield._name = "Simba" # Modifying "private" var
print(garfield.info())
```

```
$ python oop2.py
```

```
Simba is 8 years old
```

# OOP in Python / Inheritance

- A class extends another class by placing the parent class in the brackets after the class name
- To call super-constructor, must use `super().__init__(<super_args>)`
- Syntax is a little weird, but you get used to it

```
$ python oop3.py
```

```
Come here, Fido!  
Dog says Woof
```

```
class Animal():  
    def __init__(self, species, sound):  
        self.species = species  
        self.sound = sound  
  
    def roar(self):  
        print(f"{self.species} says {self.sound}")  
  
class Dog(Animal): # Dog extends from Animal  
    def __init__(self, name):  
        super().__init__("Dog", "Woof")  
        self.name = name  
  
    def call_name(self):  
        print(f"Come here, {self.name}!")  
  
fido = Dog("Fido")  
fido.call_name()  
fido.roar()
```

# OOP in Python / Dynamic patching

- Variables can be dynamically created and attached to objects
- This is actually what happens in the constructor- Notice that we don't declare the fields before assigning them in the constructor

```
class Dog():  
    def __init__(self, name):  
        self.name = name  
  
    def call_name(self):  
        print(f"Come here, {self.name}!")  
  
fido = Dog("Fido")  
fido.age = 14  
print(f"Fido is {fido.age} years old")
```

```
$ python oop4.py
```

```
Fido is 14 years old
```

# OOP in Python / Class variables

- Class variables are declared outside the constructor. These belong to the class, and not to instances of the class
- Class variables should be accessed through the class name.
- If an object has an instance variable with the same name as a static variable, the instance variable takes precedence

```
class Dog():
    fav_food = "Biscuits" #Static variable
    def __init__(self, name):
        self.name = name

fido = Dog("Fido")
print(f"1: Fido's favorite food: {fido.fav_food}")
print(f"1: All dogs favorite food: {Dog.fav_food}")

Dog.fav_food = "Treats" #Updating the static variable
print(f"2: Fido's favorite food: {fido.fav_food}")
print(f"2: All dogs favorite food: {Dog.fav_food}")

fido.fav_food = "Salad" #Updating Fido's instance variable
print(f"3: Fido's favorite food: {fido.fav_food}")
print(f"3: All dogs favorite food: {Dog.fav_food}")
```

```
$ python oop5.py
```

```
1: Fido's favorite food: Biscuits
1: All dogs favorite food: Biscuits
2: Fido's favorite food: Treats
2: All dogs favorite food: Treats
3: Fido's favorite food: Salad
3: All dogs favorite food: Treats
```

# OOP in Python / Class methods

- Class methods are declared without the self keyword.
- Class methods can only be called by prefixing the class name, not on instances of the class
- Notice the error generated: 1 positional argument given because the 'self' argument is always passed to instance methods

```
$ python oop6.py
```

```
Biscuits
```

```
Traceback (most recent call last):
```

```
File "~/oop6.py", line 12, in <module>
```

```
    fido.print_fav_food() # Not allowed
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
TypeError: Dog.print_fav_food() takes 0 positional  
arguments but 1 was given
```

```
class Dog():  
    fav_food = "Biscuits" #Static value  
    def __init__(self, name):  
        self.name = name  
  
    def call_name(self):  
        print(f"Come here, {self.name}!")  
  
    #Static method  
    def print_fav_food():  
        print(Dog.fav_food)  
  
fido = Dog("Fido")  
Dog.print_fav_food()  
fido.print_fav_food() # Not allowed  
  
# Dog.call_name() #Also not allowed
```

# Modular Programming

# Modular programming

- When developing more complex programs, not all code is in one file
- To access code in other source files it needs to be imported
- The import-statement adds the module to the current namespace
- Two types of imports:
  - Relative imports, import modules relative to the current file
  - Absolute import imports, import from the modules from the PYTHONPATH variable
- Relative imports only important within packages, otherwise always use absolute imports

```
└─ main.py  
└─ cl_dog.py
```

```
# File cl_dog.py  
class Dog():  
    def __init__(self, name):  
        self.name = name  
    def call_name(self):  
        print(f"Come here, {self.name}!")
```

```
# File main.py  
import cl_dog  
  
fido = cl_dog.Dog('Fido')  
fido.call_name()
```

```
$ python main.py  
  
Come here, Fido!
```

# Modular programming

- From <module> import <name> allows for selective importing of individual classes/functions etc.

```
# File cl_dog.py
class Dog():
    def __init__(self, name):
        self.name = name
    def call_name(self):
        print(f"Come here, {self.name}!")
```

```
# File main.py
from cl_dog import Dog

fido = Dog('Fido')
fido.call_name()
```

```
$ python main.py

Come here, Fido!
```

# Creating packages

- A python package is a collection of modules
- Structuring of modules
- `__init__.py` indicates to python that directory is a package

```
├── animals
│   ├── __init__.py
│   ├── cl_cat.py
│   └── cl_dog.py
└── main.py
```

```
# File main.py
from animals.cl_dog import Dog

fido = Dog('Fido')
fido.call_name()
```

```
$ python main.py

Come here, Fido!
```

# Packages API

- `__init__.py` lets us structure the ‘interface’ of a package
- Import classes/functions/variables as relative imports from source files
- Namespace of `__init__.py` becomes package ‘interface’
- Still possible to imports directly from modules in package

```
├── animals
│   ├── __init__.py
│   ├── cl_cat.py
│   └── cl_dog.py
└── main.py
```

```
# File __init__.py
from .cl_dog import Dog
from .cl_cat import Cat
```

```
# File main.py
from animals import Dog

fido = Dog('Fido')
fido.call_name()
```

```
$ python main.py

Come here, Fido!
```

# Using Python Environments

# Using Python Environments

- Why we use virtual environments
- How to set up a virtual environment

# Python Virtual Environments

- A virtual environment helps you install the right Python packages for a given project
- Ensures consistency and maintainability. Especially useful for multi-person teams
- If we don't use virtual environments, you and I may have different package versions. This could lead to hard-to-debug errors

# Python Virtual Environments / Setup

- Virtual environments are created using  
`python -m venv </path/to/virtual/environment>`
- The Course repo contains a script to do the installation for you, including all dependencies: `bin/venv.src.me`
  - Linux / Bash shell  
`source bin/venv.src.me`
- Once created, *source* the setup script to use the environment
  - Linux / Bash shell  
`source .venv/bin/activate`
- Your terminal will now have `(.venv)` prefixed to show that the venv is active
- To exit environment: Run `deactivate`

```
$ python -m venv .venv
$ source .venv/bin/activate
(.venv) $ #inside environment
(.venv) $ deactivate
$ #Outside the environment
```

# Python Virtual Environments / Packages

- One of the primary benefits of Python is the thousands of open-source packages that you can use in your programs
- Packages are installed with pip
  - `pip install <packagename>`
- To view installed packages
  - `pip list`
- Use a requirements file to install the right packages at the right versions
  - `pip install -r <requirements_file>`
- Course requirements in `bin/requirements.txt`
  - `venv-installation` script installs these for you

```
#requirements.txt
numpy == 1.26.4
matplotlib #No limits. Always install latest
tqdm == 4.66.2
```

```
(.venv) $ pip list #Show packages before install
```

```
Package Version
```

```
-----
pip      23.2.1
```

```
(.venv) $ pip install -r requirements.txt
```

```
# Installing ...
```

```
(.venv) $ pip list
```

```
Package      Version
```

```
-----
matplotlib   3.8.4
numpy         1.26.4
tqdm          4.66.2
```

# Python documentation

- Python has excellent documentation
- <https://docs.python.org/3.12/tutorial/index.html>
  - Goes over all the discussed concepts and more in detail
- <https://docs.python.org/3.12/index.html>
  - Links to language reference, library reference and much more